

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Facultad de Ciencias Puras · Carrera de Ingeniería Informática

INF560 — DESARROLLO WEB BACKEND

EXAMEN DE TERCER PARCIAL

Prueba práctica — Desarrollo de una API REST con Laravel Sanctum

Dato	Detalle
Materia	INF560 — Desarrollo Web Backend (5.º semestre)
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Modalidad	Práctica individual en computadora
Tema	API de gestión de una biblioteca con autenticación Sanctum
Duración sugerida	180 minutos
Puntaje total	100 puntos
Gestión	I / 2026

Instrucciones generales

- El examen es individual. Cualquier indicio de copia o uso de proyectos ajenos anula la prueba.
- Utilice Laravel 13 con PHP 8.3 o superior y PostgreSQL como motor de base de datos.
- La autenticación de la API debe implementarse obligatoriamente con Laravel Sanctum (tokens de API).
- Trabaje sobre un repositorio Git; realice commits frecuentes que evidencien su avance.
- Al finalizar, entregue el enlace del repositorio y una colección de pruebas (Postman/Insomnia) o capturas.

1. Contexto del problema

La **Biblioteca Central de la UATF** desea informatizar la gestión de su acervo y el registro de préstamos. Como parte del equipo de desarrollo backend, usted debe construir una **API REST** que exponga los servicios necesarios para autenticar usuarios, administrar el catálogo de libros y registrar los préstamos y devoluciones de ejemplares.

La API será consumida por un cliente web y por una aplicación móvil, por lo que la autenticación debe basarse en **tokens de API emitidos con Laravel Sanctum**. El sistema distingue dos tipos de usuario: el **bibliotecario**, que administra el catálogo, y el **lector**, que consulta libros y solicita préstamos.

Competencia evaluada

Desarrolla servicios web tipo API REST con Laravel, implementando autenticación por tokens con Sanctum, autorización por roles, validación de datos y lógica de negocio, aplicando buenas prácticas de diseño de endpoints y manejo de errores.

2. Requisitos técnicos

Componente	Especificación
Framework	Laravel 13
Lenguaje	PHP 8.3 o superior
Autenticación	Laravel Sanctum (API tokens)
Base de datos	PostgreSQL
Formato de respuesta	JSON (API Resources recomendado)
Control de versiones	Git + repositorio remoto (GitHub)
Pruebas	Colección Postman/Insomnia o tests con PHPUnit

3. Modelo de datos requerido

Implemente como mínimo las siguientes entidades mediante migraciones y modelos Eloquent con sus relaciones. Puede agregar campos adicionales si lo considera pertinente.

3.1 Tabla users (extendida)

Campo	Tipo	Descripción
id	bigint	Clave primaria
name	string	Nombre completo
email	string único	Correo de acceso
password	string	Contraseña con hash
role	string	Valores: bibliotecario lector

3.2 Tabla books (libros)

Campo	Tipo	Descripción
id	bigint	Clave primaria
title	string	Título del libro
author	string	Autor(a)
isbn	string único	Código ISBN
total_copies	integer	Ejemplares totales
available_copies	integer	Ejemplares disponibles

3.3 Tabla loans (préstamos)

Campo	Tipo	Descripción
id	bigint	Clave primaria
user_id	FK users	Lector que solicita el préstamo
book_id	FK books	Libro prestado
loan_date	date	Fecha del préstamo
due_date	date	Fecha límite de devolución
returned_at	date null	Fecha real de devolución
status	string	activo devuelto

Relaciones esperadas

- Un User (lector) puede tener muchos Loans (1:N).
- Un Book puede aparecer en muchos Loans (1:N).
- Cada Loan pertenece a un User y a un Book (belongsTo).

4. Endpoints solicitados

Todos los endpoints usan el prefijo /api. Las rutas marcadas como protegidas requieren un token válido de Sanctum enviado en la cabecera Authorization: Bearer <token>.

4.1 Autenticación (Sanctum)

Método	Ruta	Descripción	Acceso
POST	/api/register	Registra un usuario y devuelve su token	Pública
POST	/api/login	Valida credenciales y emite un token	Pública
POST	/api/logout	Revoca el token actual del usuario	Protegida
GET	/api/me	Devuelve los datos del usuario autenticado	Protegida

4.2 Libros

Método	Ruta	Descripción	Acceso
GET	/api/books	Lista el catálogo (con búsqueda por título)	Protegida
GET	/api/books/{id}	Muestra el detalle de un libro	Protegida
POST	/api/books	Registra un libro nuevo	Bibliotecario
PUT	/api/books/{id}	Actualiza un libro	Bibliotecario
DELETE	/api/books/{id}	Elimina un libro	Bibliotecario

4.3 Préstamos

Método	Ruta	Descripción	Acceso
GET	/api/loans	Lista todos los préstamos	Bibliotecario
GET	/api/my-loans	Lista los préstamos del usuario autenticado	Protegida
POST	/api/loans	Solicita el préstamo de un libro	Lector
PUT	/api/loans/{id}/return	Registra la devolución de un ejemplar	Protegida

Lógica de negocio obligatoria en los préstamos

- Al crear un préstamo solo se permite si available_copies > 0; en caso contrario responder 422 con un mensaje claro.
- Al crear un préstamo debe decrementarse available_copies en 1 y calcular due_date (por ejemplo, +15 días).
- Al registrar la devolución debe incrementarse available_copies, fijar returned_at y cambiar status a «devuelto».
- Un lector solo puede devolver sus propios préstamos (evitar IDOR); un bibliotecario puede gestionar cualquiera.

5. Uso obligatorio de Laravel Sanctum

Se evaluará específicamente la correcta implementación de Sanctum. Considere lo siguiente:

- Instalar y configurar Sanctum (php artisan install:api o el publicado correspondiente en Laravel 13).
- Agregar el trait HasApiTokens al modelo User.
- Emitir el token con `$user->createToken('nombre')->plainTextToken` al registrar e iniciar sesión.
- Proteger las rutas con el middleware `auth:sanctum` en `routes/api.php`.
- Revocar el token vigente en el logout con `$request->user()->currentAccessToken()->delete()`.

Como referencia, la ruta protegida mínima debe verse así:

```
// routes/api.php
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);
    Route::apiResource('books', BookController::class);
    // ... rutas de préstamos
});
```

6. Entregables y forma de entrega

- Repositorio Git con el proyecto Laravel funcional y su historial de commits.
- Migraciones ejecutables (php artisan migrate) y, opcionalmente, seeders de prueba.
- Colección de Postman/Insomnia que demuestre el flujo: registro → login → uso del token → CRUD → préstamo → devolución.
- Un archivo README.md con instrucciones de instalación y ejecución.

7. Rúbrica de evaluación

La calificación se distribuye sobre 100 puntos según los siguientes criterios. Cada criterio se evalúa como logrado, parcial o no logrado.

N.º	Criterio	Descripción de logro	Puntaje
1	Configuración del proyecto y Sanctum	Proyecto Laravel 13 funcional, Sanctum instalado y configurado, HasApiTokens en User, conexión a PostgreSQL.	10 pts
2	Migraciones, modelos y relaciones	Tablas users, books y loans con tipos correctos; relaciones Eloquent (hasMany/belongsTo) bien definidas.	15 pts
3	Registro, login y logout	register y login validan datos y emiten token; logout revoca el token; /me devuelve al usuario autenticado.	20 pts
4	CRUD de libros protegido	Endpoints de libros funcionando, protegidos con auth:sanctum, con búsqueda y respuestas JSON correctas.	20 pts
5	Gestión de préstamos y lógica de negocio	Préstamo y devolución actualizan available_copies, calculan due_date y status correctamente.	20 pts
6	Autorización por roles / abilities	Solo el bibliotecario gestiona el catálogo; el lector solicita préstamos; se evita el acceso a préstamos ajenos.	10 pts
7	Validación y manejo de errores	Uso de FormRequest o validación, códigos HTTP adecuados (401, 403, 404, 422) y mensajes claros.	5 pts

PUNTAJE TOTAL**100 pts**